Resistor Value Classifier

AI-powered classification system that identifies resistor types and values using computer vision

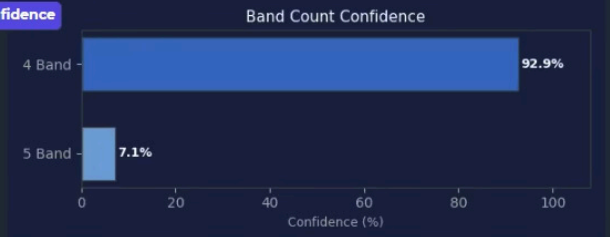
Team: The Ohmies — Jaleel Khan, Clay Farley, Andrew Rein, Ethan Messier

Classify

Step 1 — Band Count

4 Band — 92.9% confident

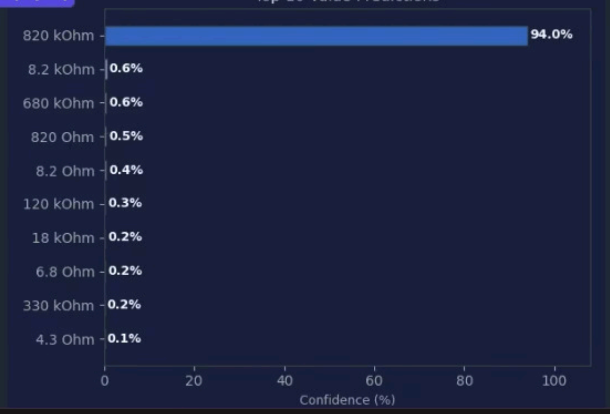
Band Count Confidence



Step 2 — Resistance Value

820 kOhm — 94.0% confident

Value Confidence (Top 10)



Predicted Color Bands

Building Resistor Identification Models
What We Have Been Working On

Last Milestone: ResNet-18 Prototype

Transfer learning approach

98.82% overall accuracy

Band Type: 100%

4-Band Value: 97%

5-Band Value: 87%

Improved ResNet-18

- Enhanced training (70 epochs)
- Smaller batches (16)
- Cleaner augmentation
- 4-Band: 98.51%
- 5-Band: 98.76%

Custom CNN

Built from scratch, no pretrained weights

- 4-Block VGG-style architecture
- ~4.8M parameters
- 4-Band: 98.38%
- 5-Band: 100%

GUI

- App to run the models in a user friendly way
  - File upload
  - Clipboard paste
  - Webcam capture
- Results Tab
- Color Band Labeling
- Classifcation History
- Confidence of Classification

Model Evolution ResNet-18: Old vs New

This comparison highlights the improvements made to the model architecture and training configuration.

More training

40% more epochs 50 → 70

Smaller batches

Half the batch size 32 → 16 for better generalization

Cleaner augmentation

Removed aggressive augmentation: brightness, blur, noise, downscale, hue/saturation

New Data Set

The new dataset is roughly **3× larger**, giving the model more diverse training data and better coverage across resistor classes.

1,270

Old Dataset

670 4-Band Resistors

600 5-Band Resistors

90 Classes

3,800

New Dataset

2,000 4-Band Resistors

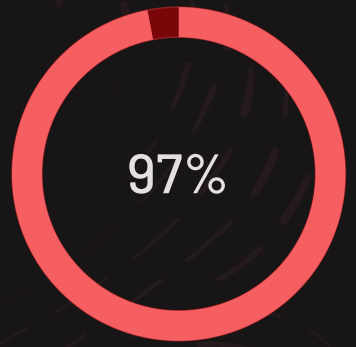
1,800 5-Band Resistors

89 Classes

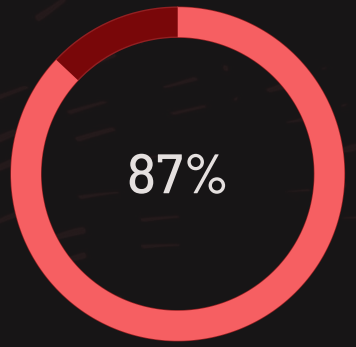
Training Results - ResNet 18

Old Version

Band Count Classifier
Validation Accuracy



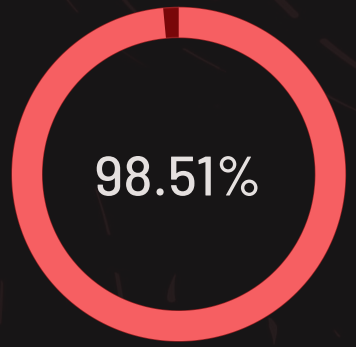
4-Band Value Classifier
Validation Accuracy



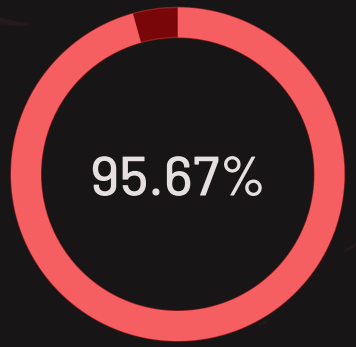
5-Band Value Classifier
Validation Accuracy

New Version

Band Count Classifier
Validation Accuracy



4-Band Value Classifier
Validation Accuracy



5-Band Value Classifier
Validation Accuracy

CNN → Three-Model Pipeline (Again)

Hierarchical CNN classification — each model specializes in one task

1

STEP 1: Band Count Classifier

Input: Any resistor photo **Output:** "4 Band" or "5 Band" **Val Accuracy: 100% Model**

2

STEP 2A: 4-Band Value Classifier

Input: 4-band image **Output:** Exact value (e.g., "820 kOhm") **Val Accuracy: 98.38% Model**

3

STEP 2B: 5-Band Value Classifier

Input: 5-band image **Output:** Exact value (e.g., "1.5 Ohm") **Val Accuracy: 100% Model**

Dataset Overview (CNN)

Two Kaggle sources with offline augmentation. Validation sets use original images only for honest evaluation metrics.

3,551
4-Band Originals

718
5-Band Originals

4,269
Total Originals

1

After Augmentation (8× per image)

- **25,352** 4-Band training images
- **7,898** 5-Band training images
- **33,250** total training images

1

Validation Sets (Originals Only)

- **79** validation images (4-band)
- **64** validation images (5-band)

Training Time (CNN) 5 ½ Hours VS Training Time (ResNet 18) 1 Hour

Model Architecture

Custom CNN built from scratch — no pretrained ImageNet weights

Block 1

Conv3×3 → BN → ReLU (×2) → MaxPool 64 filters

Block 2

Conv3×3 → BN → ReLU (×2) → MaxPool 128 filters

Block 3

Conv3×3 → BN → ReLU (×2) → MaxPool 256 filters

Block 4

Conv3×3 → BN → ReLU (×2) 512 filters

Architecture Details

~4.8M Parameters

Input: 128 × 128 × 3 RGB image

4-Block VGG-Style CNN with increasing filter counts

1

He Initialization:

Correct for ReLU, prevents vanishing/exploding gradients

2

Batch Normalization:

After every Conv layer, stabilizes training across varied lighting

3

Global Avg Pooling:

Instead of Flatten — position-invariant, 50× fewer parameters

---

Head: Dropout(0.5) → FC(256) → ReLU → Dropout(0.25) → FC(N classes)

Label Smoothing ε=0.1: Prevents overconfident predictions, improves generalization

MixUp Augmentation: Blends two training images — forces learning generalizable features

Resistor CNN Code Walkthrough

Model Architecture (ResistorCNN)

- VGG-style backbone built from repeated `_conv_block` patterns
- Progressive channel deepening: 3 → 64 → 128 → 256 → 512
- Global Average Pooling reduces spatial features before classification
- Shallow classifier head with He initialization for stable training

Stage 1: Band Count Classifier

- Binary classifier that distinguishes 4-band from 5-band resistors
- Careful train/validation split with stratification to preserve class balance
- `WeightedRandomSampler` handles class imbalance during training
- `CosineAnnealingLR` supports smooth learning-rate decay
- Label smoothing improves generalization
- Gradient clipping stabilizes optimization
- Early stopping prevents overfitting and saves the best checkpoint

Stage 2: Value Classifiers

- Two-phase transfer learning strategy for resistance prediction
- Phase 1: train the classification head only
- Phase 2: fine-tune the full network with MixUp augmentation
- Problem-specific mapping converts predictions into resistance values

Design Philosophy

- Coarse-to-fine cascade first routes by band count, then resolves resistance value
- Lightweight binary gatekeeper keeps the first decision fast and efficient
- Specialized value classifiers reuse transferred low-level features for better accuracy

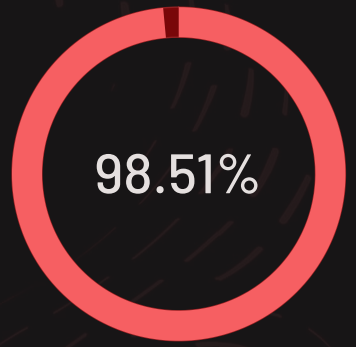
```python
class ResistorCNN(nn.Module):
    """
    4-block convolutional network for 4-band vs 5-band resistor classification.

    Input:  (N, 3, 128, 128)   RGB image batch
    Output: (N, num_classes)   raw logits (apply softmax for probabilities)
    """
    def __init__(self, num_classes: int = 2, dropout1: float = 0.5,
                 dropout2: float = 0.25):
        super().__init__()
        # ── Feature extractor ──────────────────────────────────────────
        self.features = nn.Sequential(
            _conv_block(3,   64),    # 128x128 → 64x64
            _conv_block(64,  128),   #  64x64  → 32x32
            _conv_block(128, 256),   #  32x32  → 16x16
            _conv_block(256, 512),   #  16x16  →  8x8
        )
        # Global Average Pool: (N, 512, 8, 8) → (N, 512)
        self.gap = nn.AdaptiveAvgPool2d(1)
        # ── Classifier head ────────────────────────────────────────────
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout1),
            nn.Linear(512, 256),
            nn.ReLU(inplace=False),
            nn.Dropout(p=dropout2),
            nn.Linear(256, num_classes),
        )
        # He initialization
        self._init_weights()
    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out',
                                        nonlinearity='relu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight)
                nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
                nn.init.zeros_(m.bias)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.gap(x)
        x = x.flatten(1)          # (N, 512)
        x = self.classifier(x)    # (N, num_classes)
        return x
    def get_feature_map(self, x: torch.Tensor) -> torch.Tensor:
        """Return the final conv block output (used for Grad-CAM)."""
        return self.features(x)
```

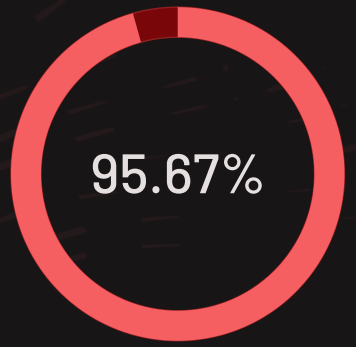
Training Results CNN & ResNet 18

ResNet 18

Band Count Classifier
Validation Accuracy



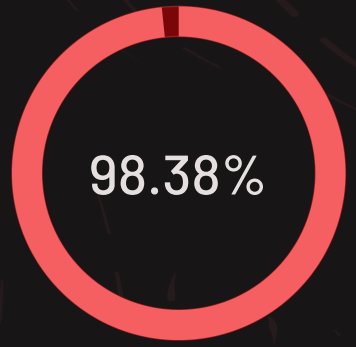
4-Band Value Classifier
Validation Accuracy



5-Band Value Classifier
Validation Accuracy

CNN

Band Count Classifier
Validation Accuracy



4-Band Value Classifier
Validation Accuracy

5-Band Value Classifier
Validation Accuracy

High-Level Code Overview — Resistor Value Classifier

data/ → augment scripts → training scripts → checkpoints/ → app.py → browser

model.py

Architecture

› Defines ResistorCNN class used by ALL 3 models

› 4-block VGG-style CNN · ~4.8M parameters

› Only num_classes changes per checkpoint

› He init · BN · GAP · Dropout head

dataset.py

Constants

› Defines MEAN, STD, IMG_SIZE (128)

› Shared by app.py AND all train scripts

› Changing these affects all transforms

› Normalization must match train ↔ inference

parse_folder_name.py

Label Parser

› Converts folder name → resistance label

› 4B-820K-T5 → "820 kOhm"

› Strips tolerance · handles decimals

› Merges duplicate values across datasets

augment_4/5band.py

Data Multiplier

› One-time offline scripts · safe to re-run

› Skips files that already exist

› 4k originals → 33k augmented images

› Rotation · brightness · warp · flip · crop

train.py

Band Count Trainer

› Trains best_model.pth (2 classes)

› Reads data/4 Band/ and data/5 Band/

› Single-phase · cosine annealing

› Stops at 100% val acc or patience=15

4BandModel.py / 5BandModel.py

Value Trainers

› Train value classifier checkpoints

› Phase 1: head only (lr=1e-3, 15–20 ep)

› Phase 2: full fine-tune + MixUp (80–120 ep)

› Loads best_model.pth as starting weights

app.py

Main Application

› Loads all 3 checkpoints on startup

› classify_resistor(): full TTA pipeline

› Routes band count → correct value model

› Gradio: 5 tabs · webcam · history · Grad-CAM

predict.py / evaluate.py

Utilities

› predict.py: single-image CLI prediction

› evaluate.py: test set metrics + confusion matrix

› Quick sanity check without launching app

› Used to generate poster accuracy numbers

Dependency tree: app.py → model.py · dataset.py · checkpoints/ 4/5BandModel.py → model.py · dataset.py · parse_folder_name.py · data/

Band Count Classification

Step 1: Identifies 4-band vs 5-band with confidence chart

Resistance Value Prediction

Step 2: Routes to correct value model, outputs exact Ω value

Top 3 Predictions

Shows top 3 candidates with confidence percentages

Session History

Running log of last 10 classifications with timestamps

Confidence Warnings

Alerts user when confidence is below 60% threshold

Webcam Support

Camera capture directly in the browser interface

Color Band Diagram

Draws the predicted resistor color bands visually for verification

Grad-CAM Visualization

Heatmap showing exactly what region the model focused on

What's Next?

Stand Alone Embedded deployment on Raspberry Pi 5 with Camera & Comparison

1

Completed Milestones

- ResNet-18 prototype
- ResNet-18 Improved
- Custom CNN architecture built from scratch
- GUI

Phase 1 — CPU Inference

Hardware: Raspberry Pi 5, Pironman 5 Pro Max case

- Port PyTorch models to RPi5 ARM
- Run app.py on Pi browser
- Replace webcam input with Pi Camera Module feed

Phase 2 — Hailo8 Acceleration

- Convert .pth → ONNX → Hailo .hef format
- 26 TOPS inference for real-time performance
- Full-screen Gradio on 4.3" display, touch-optimized layout
- Self-contained resistor scanner — no PC required

4

Head-to-Head Benchmark

- Run 50 unseen resistor images through both model types
- Compare prediction accuracy, confidence scores, and failure cases head-to-head
- Find which model worked the best

Training Configuration (Extra)

Two-phase transfer learning approach for value classifiers

Optimizer & Schedule

- Adam optimizer (lr = 3e-4)
- Cosine annealing LR schedule
- Gradient clipping (max norm = 1.0)
- Weight decay = 1e-4
- Label smoothing ε = 0.1

Two-Phase Fine-Tuning

Phase 1 — Head only (15-20 epochs): Conv blocks frozen, only output layer trains (lr = 1e-3)

Phase 2 — Full fine-tune (80-120 epochs): All layers unfrozen (lr = 3e-4), MixUp augmentation enabled, early stop at 100% or patience=20

Input & Hardware

- Input: 128 × 128 × 3 RGB
- Batch size: 32
- Weighted random sampler (corrects class imbalance)
- GPU: NVIDIA RTX 3080 Ti
- CUDA 12.1 / PyTorch 2.1
- TTA at inference (8 passes)